Code walkthrough

tb_alu.v

Testbench: ALU

What it does	Checks every ALU operation (11 tests)
Where it works	module-level test
Size	73 lines, explained in 6 small parts
Modules in this file	tb_alu
Calls (uses)	alu.v (module alu)
Called by	nobody - this is a testbench (top of simulation)

A testbench is simulation-only code that drives inputs into a design and checks the outputs. This one tests the ALU on its own. The ALU has no clock, so the testbench just sets inputs, waits a moment and compares.

Map of the file

Part	Lines	What this part is
1	1-11	Time unit and header
2	12-20	Signals
3	22-24	The design under test (DUT)
4	26-49	The test task
5	51-65	The 11 tests
6	67-73	Summary

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Time unit and header (lines 1-11)

tb_alu.v lines 1-11

```
 1  `timescale 1ns / 1ps
 2  //
 3  // CONNECTIONS
 4  //   Calls       : alu.v (module alu)
 5  //   Called by   : nobody - this is a testbench (top of simulation)
 6  // ================================================================
 7  // FILE: tb_alu.v
 8  // MODULE-LEVEL TEST 1: the ALU on its own (no clock needed).
 9  // Covers every ALU operation in the project plan, plus the Zero flag
10  // and the Carry / no-borrow convention.
11  // ================================================================
```

- `` `timescale 1ns / 1ps ``: #1 means 1 nanosecond. The header says what is tested.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Signals (lines 12-20)

tb_alu.v lines 12-20

```
12  module tb_alu;
13
14      reg  [3:0] a, b;
15      reg  [2:0] alu_ctrl;
16      wire [3:0] result;
17      wire       carry, zero;
18
19      integer errors = 0;
20      integer tests  = 0;
```

- A testbench has no ports. `reg` signals are the inputs we drive; `wire` signals are the outputs we read.
- errors and tests count the results.

Part 3 · The design under test (DUT) (lines 22-24)

tb_alu.v lines 22-24

```
22      // CALLS: module alu from file alu.v
23      alu DUT (.a(a), .b(b), .alu_ctrl(alu_ctrl),
24               .result(result), .carry(carry), .zero(zero));
```

One copy of the ALU, connected to the testbench signals.

→ This part **calls module** `alu` from file `alu.v`. How that module works is explained in `Explained_alu.pdf`.

Part 4 · The test task (lines 26-49)

tb_alu.v lines 26-49

```
26      // apply one test and compare with the expected values
27      task test;
28          input [8*12:1] name;
29          input [3:0]    in_a, in_b;
30          input [2:0]    op;
31          input [3:0]    exp_result;
32          input          exp_carry;
33          input          exp_zero;
34          begin
35              a = in_a;  b = in_b;  alu_ctrl = op;
36              #10;
37              tests = tests + 1;
38              if (result == exp_result && carry == exp_carry && zero == exp_zero)
39                  $display("PASS  %0s a=%b b=%b -> result=%b carry=%b zero=%b",
40                           name, a, b, result, carry, zero);
41              else begin
42                  $display("FAIL  %0s a=%b b=%b -> result=%b carry=%b zero=%b",
43                           name, a, b, result, carry, zero);
44                  $display("      expected result=%b carry=%b zero=%b",
45                           exp_result, exp_carry, exp_zero);
46                  errors = errors + 1;
47              end
48          end
49      endtask
```

- A task is a reusable procedure. It sets a, b and alu_ctrl, waits 10 ns (#10), then compares result, carry and zero with the expected values.
- It prints PASS or FAIL; on FAIL it also prints what was expected and adds 1 to errors.
- %0s prints the test name, %b prints in binary.

Part 5 · The 11 tests (lines 51-65)

tb_alu.v lines 51-65

```
51      initial begin
52          // The task 'test' used below is defined above in this same file.
53          $display("\n===== tb_alu: ALU module test =====");
54          //      name            a        b        ctrl  result   C  Z
55          test("ADD 5+3     ", 4'b0101, 4'b0011, 3'd0, 4'b1000, 0, 0);
56          test("ADD 15+1    ", 4'b1111, 4'b0001, 3'd0, 4'b0000, 1, 1);
57          test("SUB 5-3     ", 4'b0101, 4'b0011, 3'd1, 4'b0010, 1, 0);
58          test("SUB 3-5     ", 4'b0011, 4'b0101, 3'd1, 4'b1110, 0, 0);
59          test("SUB 7-7     ", 4'b0111, 4'b0111, 3'd1, 4'b0000, 1, 1);
60          test("AND         ", 4'b1100, 4'b1010, 3'd2, 4'b1000, 0, 0);
61          test("OR          ", 4'b1100, 4'b1010, 3'd3, 4'b1110, 0, 0);
62          test("XOR         ", 4'b1100, 4'b1010, 3'd4, 4'b0110, 0, 0);
63          test("XOR to zero ", 4'b0110, 4'b0110, 3'd4, 4'b0000, 0, 1);
64          test("NOT 0101    ", 4'b0101, 4'b0000, 3'd5, 4'b1010, 0, 0);
65          test("PASS b      ", 4'b0000, 4'b1001, 3'd6, 4'b1001, 0, 0);
```

Each line is one test: name, a, b, ALU code, expected result, expected Carry, expected Zero. They cover ADD (normal and overflow), SUB (no borrow, borrow, zero result), AND, OR, XOR, XOR to zero, NOT and PASS.

Part 6 · Summary (lines 67-73)

tb_alu.v lines 67-73

```
67          if (errors == 0)
68              $display("tb_alu PASSED: %0d of %0d tests OK\n", tests, tests);
69          else
70              $display("tb_alu FAILED: %0d of %0d tests failed\n", errors, tests);
71          $finish;
72      end
73  endmodule
```

Prints PASSED or FAILED with the counts, then $finish stops the simulation.

> **In short:** 11 one-line tests, all passing.